

Machine Learning: Generative Learning Algorithms

Quoc Kien

1. Triết lý Mô hình Phân biệt (Discriminative) vs. Mô hình Sinh (Generative)

Khi giải quyết một bài toán phân loại, hai hướng tiếp cận cốt lõi bao gồm:

- **Mô hình Phân biệt (Discriminative Models):** Học trực tiếp xác suất điều kiện $P(y | \mathbf{x})$ từ dữ liệu (ví dụ: Logistic Regression). Mô hình này cố gắng tìm ra một đường ranh giới tối ưu để phân tách các lớp dữ liệu mà không cần quan tâm đến việc dữ liệu đó được sinh ra như thế nào.
- **Mô hình Sinh (Generative Models):** Thay vì học trực tiếp ranh giới, mô hình sinh đi tìm hiểu cách từng lớp dữ liệu được tạo ra một cách độc lập bằng cách mô hình hóa xác suất đồng thời (Joint Probability):

$$P(\mathbf{x}, y) = P(\mathbf{x} | y)P(y)$$

Sau khi đã học được phân phối $P(\mathbf{x} | y)$ cho từng lớp và xác suất tiên nghiệm (Prior) $P(y)$, mô hình áp dụng **Định lý Bayes** để tính xác suất hậu nghiệm (Posterior) tại giai đoạn dự đoán:

$$P(y = 1 | \mathbf{x}) = \frac{P(\mathbf{x} | y = 1)P(y = 1)}{P(\mathbf{x})}$$

Trong đó, mẫu số được khai triển bằng công thức xác suất đầy đủ:

$$P(\mathbf{x}) = P(\mathbf{x} | y = 1)P(y = 1) + P(\mathbf{x} | y = 0)P(y = 0)$$

Tùy thuộc vào bản chất của các thuộc tính đầu vào $\mathbf{x}$, ta có hai thuật toán sinh kinh điển: 1. **Thuộc tính liên tục ($\mathbf{x} \in \mathbb{R}^p$):** Gaussian Discriminant Analysis (GDA). 2. **Thuộc tính rời rạc:** Naive Bayes.

2. Phân tích Phân biệt Gaussian (Gaussian Discriminant Analysis - GDA)

GDA giả định rằng dữ liệu đầu vào ứng với mỗi nhãn cụ thể tuân theo một **Phân phối Chuẩn Đa Biến (Multivariate Gaussian Distribution)**.

2.1 Nhắc lại kiến thức: Phân phối Chuẩn Đa Biến

Một vector ngẫu nhiên $\mathbf{x} \in \mathbb{R}^p$ tuân theo phân phối chuẩn đa biến với vector kỳ vọng $\mu \in \mathbb{R}^p$ và ma trận hiệp phương sai $\Sigma \in \mathbb{R}^{p \times p}$ (ký hiệu $\mathbf{x} \sim \mathcal{N}(\mu, \Sigma)$) có hàm mật độ xác suất là:

$$P(\mathbf{x}; \mu, \Sigma) = \frac{1}{(2\pi)^{p/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mu)^T \Sigma^{-1} (\mathbf{x} - \mu) \right)$$

Trong đó $|\Sigma|$ là định thức của ma trận Σ .

2.2 Các giả định nền tảng của GDA

Xét bài toán phân loại nhị phân ($y \in \{0, 1\}$). GDA áp đặt các giả định phân phối sau: * Nhãn mục tiêu y tuân theo phân phối Bernoulli với tham số ϕ :

$$y \sim \text{Bernoulli}(\phi) \implies P(y) = \phi^y (1 - \phi)^{1-y}$$

* Dữ liệu đầu vào điều kiện theo lớp $y = 0$ tuân theo phân phối chuẩn:

$$\mathbf{x} \mid y = 0 \sim \mathcal{N}(\mu_0, \Sigma)$$

* Dữ liệu đầu vào điều kiện theo lớp $y = 1$ tuân theo phân phối chuẩn:

$$\mathbf{x} \mid y = 1 \sim \mathcal{N}(\mu_1, \Sigma)$$

Lưu ý quan trọng: GDA giả định các lớp có vector kỳ vọng riêng biệt ($\mu_0 \neq \mu_1$) nhưng **chia sẻ chung một ma trận hiệp phương sai Σ** . Điều này giúp ranh giới quyết định toán học sau này là một đường thẳng (tuyến tính).

3. Ước lượng Tham số GDA bằng Maximum Likelihood Estimation (MLE)

Để tìm các tham số $\{\phi, \mu_0, \mu_1, \Sigma\}$, ta thiết lập hàm Log-Likelihood đồng thời trên toàn bộ n mẫu dữ liệu huấn luyện:

$$\ell(\phi, \mu_0, \mu_1, \Sigma) = \sum_{i=1}^n \log P(\mathbf{x}^{(i)}, y^{(i)}) = \sum_{i=1}^n [\log P(\mathbf{x}^{(i)} | y^{(i)}) + \log P(y^{(i)})]$$

Khai triển chi tiết hàm mục tiêu tối ưu hóa:

$$\ell = \sum_{i=1}^n \left[\log \left(\frac{1}{(2\pi)^{p/2} |\Sigma|^{1/2}} \right) - \frac{1}{2} (\mathbf{x}^{(i)} - \mu_{y^{(i)}})^T \Sigma^{-1} (\mathbf{x}^{(i)} - \mu_{y^{(i)}}) + y^{(i)} \log \phi + (1 - y^{(i)}) \log(1 - \phi) \right]$$

3.1 Ước lượng Tham số Tiên nghiệm ϕ

Lấy đạo hàm riêng của ℓ theo biến vô hướng ϕ và đặt bằng 0:

$$\frac{\partial \ell}{\partial \phi} = \sum_{i=1}^n \left[\frac{y^{(i)}}{\phi} - \frac{1 - y^{(i)}}{1 - \phi} \right] = 0$$

$$\sum_{i=1}^n \frac{y^{(i)} - \phi}{\phi(1 - \phi)} = 0 \implies \sum_{i=1}^n y^{(i)} - n\phi = 0 \implies \phi = \frac{1}{n} \sum_{i=1}^n y^{(i)}$$

3.2 Ước lượng các Vector Kỳ vọng μ_0 và μ_1

Để tìm μ_0 , ta chỉ quan tâm đến các số hạng chứa μ_0 (các mẫu có $y^{(i)} = 0$). Áp dụng quy tắc đạo hàm ma trận $\frac{\partial}{\partial \mathbf{z}} (\mathbf{a} - \mathbf{z})^T \mathbf{A} (\mathbf{a} - \mathbf{z}) = -2\mathbf{A}(\mathbf{a} - \mathbf{z})$:

$$\nabla_{\mu_0} \ell = \sum_{i: y^{(i)}=0} \Sigma^{-1} (\mathbf{x}^{(i)} - \mu_0) = \mathbf{0}$$

$$\Sigma^{-1} \sum_{i: y^{(i)}=0} (\mathbf{x}^{(i)} - \mu_0) = \mathbf{0} \implies \mu_0 = \frac{\sum_{i=1}^n (1 - y^{(i)}) \mathbf{x}^{(i)}}{\sum_{i=1}^n (1 - y^{(i)})}$$

Tương tự cho lớp 1, ta có công thức nghiệm đóng:

$$\mu_1 = \frac{\sum_{i=1}^n y^{(i)} \mathbf{x}^{(i)}}{\sum_{i=1}^n y^{(i)}}$$

3.3 Ước lượng Ma trận Hiệp phương sai Chung Σ

Bằng phép toán tối ưu hóa ma trận nâng cao, ta thu được ma trận hiệp phương sai thực nghiệm mẫu:

$$\Sigma = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}^{(i)} - \mu_{y^{(i)}})(\mathbf{x}^{(i)} - \mu_{y^{(i)}})^T$$

4. Chứng minh GDA tương đương toán học với Hàm Sigmoid

Một điểm bộc phá lý thuyết đỉnh cao là: Nếu dữ liệu thỏa mãn các giả định của GDA, thì xác suất hậu nghiệm $P(y = 1 | \mathbf{x})$ được suy diễn ra sẽ có dạng chính xác là một **Hàm số Sigmoid**.

Biến đổi Đại số chi tiết

Áp dụng Định lý Bayes:

$$P(y = 1 | \mathbf{x}) = \frac{P(\mathbf{x} | y = 1)P(y = 1)}{P(\mathbf{x} | y = 1)P(y = 1) + P(\mathbf{x} | y = 0)P(y = 0)} = \frac{1}{1 + \frac{P(\mathbf{x}|y=0)P(y=0)}{P(\mathbf{x}|y=1)P(y=1)}}$$

Ta viết lại biểu thức dưới dạng hàm mũ:

$$P(y = 1 | \mathbf{x}) = \frac{1}{1 + \exp\left(-\log\left(\frac{P(\mathbf{x}|y=1)P(y=1)}{P(\mathbf{x}|y=0)P(y=0)}\right)\right)}$$

Đặt giá trị bên trong hàm mũ là số hạng z :

$$z = \log\left(\frac{P(\mathbf{x} | y = 1)}{P(\mathbf{x} | y = 0)}\right) + \log\left(\frac{P(y = 1)}{P(y = 0)}\right)$$

Thay hàm mật độ xác suất chuẩn đa biến vào thành phần thứ nhất của z :

$$\log\left(\frac{P(\mathbf{x} | y = 1)}{P(\mathbf{x} | y = 0)}\right) = \log\left(\frac{\frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu_1)^T \Sigma^{-1}(\mathbf{x} - \mu_1)\right)}{\frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu_0)^T \Sigma^{-1}(\mathbf{x} - \mu_0)\right)}\right)$$

Triệt tiêu hằng số chuẩn hóa mặt trước và lấy logarit tự nhiên để triệt tiêu hàm số exp:

$$= -\frac{1}{2}(\mathbf{x} - \mu_1)^T \Sigma^{-1}(\mathbf{x} - \mu_1) + \frac{1}{2}(\mathbf{x} - \mu_0)^T \Sigma^{-1}(\mathbf{x} - \mu_0)$$

Khai triển các đa thức ma trận (lưu ý rằng $\mathbf{x}^T \Sigma^{-1} \mu_1 = \mu_1^T \Sigma^{-1} \mathbf{x}$ vì đây là đại lượng vô hướng):

$$= -\frac{1}{2} [\mathbf{x}^T \Sigma^{-1} \mathbf{x} - 2\mu_1^T \Sigma^{-1} \mathbf{x} + \mu_1^T \Sigma^{-1} \mu_1] + \frac{1}{2} [\mathbf{x}^T \Sigma^{-1} \mathbf{x} - 2\mu_0^T \Sigma^{-1} \mathbf{x} + \mu_0^T \Sigma^{-1} \mu_0]$$

Ta thấy số hạng bậc hai $\mathbf{x}^T \Sigma^{-1} \mathbf{x}$ bị triệt tiêu hoàn toàn (nhờ giả định chung ma trận Σ). Gom các số hạng còn lại theo biến $\mathbf{x}$:

$$= (\mu_1^T \Sigma^{-1} - \mu_0^T \Sigma^{-1}) \mathbf{x} - \frac{1}{2} \mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2} \mu_0^T \Sigma^{-1} \mu_0$$

Kết hợp với thành phần tiên nghiệm $\log\left(\frac{\phi}{1-\phi}\right)$, toàn bộ biểu thức z có cấu trúc tuyến tính hoàn hảo:

$$z = \theta^T \mathbf{x} + \theta_0$$

Trong đó: $\theta = \Sigma^{-1}(\mu_1 - \mu_0)$ $\theta_0 = -\frac{1}{2}\mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2}\mu_0^T \Sigma^{-1} \mu_0 + \log \frac{\phi}{1-\phi}$

Hệ quả là:

$$P(y = 1 | \mathbf{x}) = \frac{1}{1 + e^{-(\theta^T \mathbf{x} + \theta_0)}} = \sigma(\theta^T \mathbf{x} + \theta_0)$$

5. Thuật toán Phân loại Naive Bayes (Dữ liệu rời rạc)

Khi các đặc trưng đầu vào $\mathbf{x}$ là các biến rời rạc định danh (ví dụ: bài toán lọc thư rác từ các từ vựng xuất hiện), giả định phân phối liên tục Gauss không còn phù hợp. Ta chuyển sang sử dụng **Naive Bayes**.

5.1 Giả định Độc lập Điều kiện Naive Bayes

Nếu không có giả định đơn giản hóa, để mô hình hóa một vector thuộc tính nhị phân $\mathbf{x} \in \{0, 1\}^p$, ta cần một bảng xác suất khổng lồ chứa $2^p - 1$ tham số, điều này gây bùng nổ dữ liệu toán học.

Giả định “Ngây thơ” (Naive) đặt ra là: **Các đặc trưng đầu vào x_j độc lập với nhau khi biết trước nhãn lớp y .**

$$P(x_1, x_2, \dots, x_p | y) = \prod_{j=1}^p P(x_j | y)$$

5.2 Thiết lập các Tham số và MLE

Xét bài toán phân loại văn bản (Text Classification) với từ điển kích thước p . Đặt: $\phi_{j|y=1} = P(x_j = 1 | y = 1)$ $\phi_{j|y=0} = P(x_j = 1 | y = 0)$ $\phi_y = P(y = 1)$

Hàm Log-Likelihood đồng thời trên toàn bộ tập dữ liệu huấn luyện:

$$\ell(\phi_y, \phi_{j|y=1}, \phi_{j|y=0}) = \sum_{i=1}^n \log \left(\prod_{j=1}^p P(x_j^{(i)} | y^{(i)}) \cdot P(y^{(i)}) \right)$$

Tối ưu hóa hàm mục tiêu bằng phương pháp MLE, ta thu được các tỷ lệ đếm thực nghiệm trực quan:

$$\begin{aligned}\phi_{j|y=1} &= \frac{\sum_{i=1}^n \mathbb{I}(x_j^{(i)} = 1 \wedge y^{(i)} = 1)}{\sum_{i=1}^n \mathbb{I}(y^{(i)} = 1)} \\ \phi_{j|y=0} &= \frac{\sum_{i=1}^n \mathbb{I}(x_j^{(i)} = 1 \wedge y^{(i)} = 0)}{\sum_{i=1}^n \mathbb{I}(y^{(i)} = 0)} \\ \phi_y &= \frac{\sum_{i=1}^n \mathbb{I}(y^{(i)} = 1)}{n}\end{aligned}$$

5.3 Kỹ thuật mịn hóa Laplace (Laplace Smoothing)

Nếu một từ vựng mới chưa từng xuất hiện trong lớp 1 ở tập huấn luyện, giá trị đếm bằng 0 dẫn đến $\phi_{j|y=1} = 0$. Khi gặp văn bản mới chứa từ này, phép nhân tích chuỗi sẽ triệt tiêu toàn bộ xác suất hậu nghiệm xuống bằng 0 một cách sai lầm ($P(y = 1 | \mathbf{x}) = 0$).

Để khắc phục, kỹ thuật **Laplace Smoothing** cộng thêm 1 vào tử số và cộng thêm kích thước miền giá trị vào mẫu số:

$$\phi_{j|y=1} = \frac{\sum_{i=1}^n \mathbb{I}(x_j^{(i)} = 1 \wedge y^{(i)} = 1) + 1}{\sum_{i=1}^n \mathbb{I}(y^{(i)} = 1) + 2}$$

6. Thực hành Triển khai GDA bằng Python từ đầu (Scratch)

Đoạn mã Python dưới đây hiện thực hóa toàn bộ các bước tính toán MLE của thuật toán GDA và dự đoán xác suất hậu nghiệm:

```
import numpy as np
import matplotlib.pyplot as plt

class GaussianDiscriminantAnalysis:
    def __init__(self):
        self.phi = None
        self.mu0 = None
        self.mu1 = None
        self.sigma = None

    def fit(self, X, y):
        n_samples, n_features = X.shape

        # 1. Ước lượng tham số Bernoulli phi
        self.phi = np.mean(y)

        # 2. Phân tách các mẫu dữ liệu theo từng lớp
        X0 = X[y == 0]
        X1 = X[y == 1]

        # 3. Tính toán các vector kỳ vọng mu0 và mu1
        self.mu0 = np.mean(X0, axis=0)
        self.mu1 = np.mean(X1, axis=0)

        # 4. Tính toán ma trận hiệp phương sai chung mẫu shared sigma
        dev0 = X0 - self.mu0
        dev1 = X1 - self.mu1

        # Cộng gộp tổng bình phương độ lệch trên toàn bộ dữ liệu
        shared_covariance = (np.dot(dev0.T, dev0) + np.dot(dev1.T, dev1)) /
        ↪ n_samples
        self.sigma = shared_covariance

    def _multivariate_gaussian(self, X, mu, sigma):
        # Tính toán hàm mật độ xác suất chuẩn đa biến Gauss
        p = len(mu)
        det_sigma = np.linalg.det(sigma)
        inv_sigma = np.linalg.inv(sigma)
```

```

# Tính hằng số chuẩn hóa mặt trước
norm_const = 1.0 / ((2 * np.pi) ** (p / 2) * np.sqrt(det_sigma))

# Tính khoảng cách Mahalanobis trong hàm mũ
dev = X - mu
exponent = -0.5 * np.sum(np.dot(dev, inv_sigma) * dev, axis=1)
return norm_const * np.exp(exponent)

def predict_proba(self, X):
    # Giai đoạn dự đoán áp dụng Định lý Bayes
    px_y0 = self._multivariate_gaussian(X, self.mu0, self.sigma)
    px_y1 = self._multivariate_gaussian(X, self.mu1, self.sigma)

    # Tính xác suất đồng thời joint probability
    joint_y0 = px_y0 * (1 - self.phi)
    joint_y1 = px_y1 * self.phi

    # Chuẩn hóa posterior
    posterior_y1 = joint_y1 / (joint_y0 + joint_y1)
    return posterior_y1

# Kiểm thử thuật toán với dữ liệu giả lập mẫu
np.random.seed(42)
X_data = np.vstack([np.random.multivariate_normal([1.0, 1.0], [[1, 0], [0,
↪ 1]], 50),
                    np.random.multivariate_normal([3.0, 4.0], [[1, 0], [0,
↪ 1]], 50)])
y_data = np.hstack([np.zeros(50), np.ones(50)])

gda = GaussianDiscriminantAnalysis()
gda.fit(X_data, y_data)
print("Vector mu0 thực nghiệm tối ưu của GDA:", gda.mu0)
print("Vector mu1 thực nghiệm tối ưu của GDA:", gda.mu1)
print("Estimated shared covariance:")
print(np.round(gda.sigma, 3))

xx, yy = np.meshgrid(np.linspace(-2, 6, 160), np.linspace(-2, 7, 160))
grid = np.c_[xx.ravel(), yy.ravel()]
posterior = gda.predict_proba(grid).reshape(xx.shape)

plt.figure(figsize=(7, 5))
contour = plt.contourf(xx, yy, posterior, levels=20, cmap="RdBu_r",
↪ alpha=0.75)

```



```

plt.colorbar(contour, label="$P(y=1 \mid x)$")
plt.scatter(X_data[y_data == 0, 0], X_data[y_data == 0, 1], label="class 0",
    ↪ edgecolor="black", alpha=0.85)
plt.scatter(X_data[y_data == 1, 0], X_data[y_data == 1, 1], label="class 1",
    ↪ edgecolor="black", alpha=0.85)
plt.contour(xx, yy, posterior, levels=[0.5], colors="black", linewidths=2)
plt.title("GDA posterior surface and decision boundary")
plt.xlabel("$x_1$")
plt.ylabel("$x_2$")
plt.legend()
plt.grid(alpha=0.2)
plt.show()

```

Vector μ_0 thực nghiệm tối ưu của GDA: [0.86432437 0.9279826]

Vector μ_1 thực nghiệm tối ưu của GDA: [2.90454712 4.14006205]

Estimated shared covariance:

```

[[0.726 0.025]
 [0.025 0.976]]

```

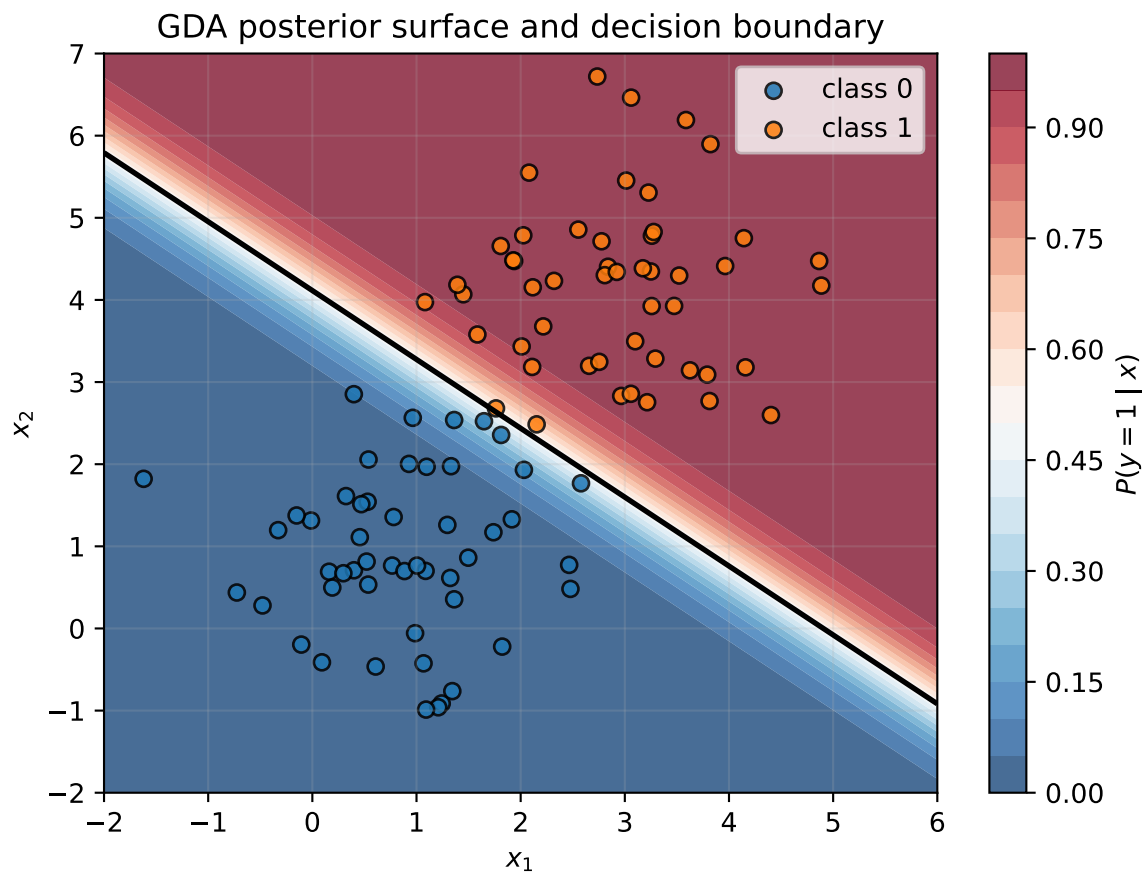


Figure 1: GDA posterior probabilities learned from two Gaussian-like classes.